

Informe técnico: Agente generador de informes con ADK

1. Introducción

En este proyecto hemos desarrollado un agente inteligente generador de documentos que a partir de búsquedas en la web genera un documento PDF y un JSON con sus estadísticas. Hemos utilizado Google ADK para la implementación del sistema y LiteLLM para integrar el modelo de lenguaje.

El objetivo principal es que a partir de una consulta del usuario, el agente siga un flujo de trabajo que resulte en la producción de estos dos documentos siguiendo una estructura determinada.

2. Arquitectura del sistema

El componente principal del sistema es el **root_agent**, que conectándose a un modelo LLM a través de LiteLLM, se encarga de dirigir la ejecución de todo el proceso, desde la búsqueda de información hasta la generación del informe. Esto lo hace usando ciertas herramientas.

El sistema incluye tres herramientas principales:

- **web_search_tool**: Realiza búsquedas en Wikipedia y DuckDuckGo para obtener fuentes relevantes sobre el tema. Devuelve una lista estructurada con títulos, URLs y tipo de fuente. Si no encuentra fuentes relevantes, devuelve un mensaje de error.
- **fetch_content_tool**: Extrae el contenido textual detallado de una URL específica utilizando la API de Wikipedia o la librería traifatura para páginas web. Usa esta herramienta sobre las mejores fuentes encontradas en **web_search_tool**.
- **save_report_artifacts**: Recibe un informe estructurado, valida y normaliza su contenido y genera los artefactos finales (PDF y JSON) llamando internamente a las funciones correspondientes. Es la herramienta obligatoria de salida del agente.

Además, existen dos funciones auxiliares:

- **create_pdf_report**: Genera un documento PDF en la carpeta output del directorio del proyecto usando la clase FPDF de Python. Este documento estará estructurado en: Título, Secciones y Bibliografía.
- **create_json_report**: Crea el archivo JSON con estadísticas del informe, en la misma carpeta donde se guardará el PDF. Estas estadísticas incluyen: El título del documento, cuantas palabras tiene cada sección, el número total de palabras, secciones y referencias, y finalmente la ruta del pdf generado.

Esta separación permite distinguir entre la lógica del agente y la de la generación de archivos.

3. Flujo de ejecución

El funcionamiento del sistema sigue los siguientes pasos:

1. El usuario introduce un tema técnico.
2. El agente llama a **web_search_tool** para obtener fuentes relevantes.
3. Selecciona un máximo de 10 URLs y utiliza **fetch_content_tool** para extraer contenido.
4. A partir de la información obtenida, redacta un informe técnico en español.

5. Construye un objeto estructurado con título, secciones y referencias.
6. Llama a la herramienta `save_report_artifacts`.
7. Esta herramienta genera el PDF y el JSON en la carpeta output.
8. El agente devuelve la ruta real de los archivos generados.

4. Decisiones de diseño

La encapsulación de la generación de los documentos en `save_report_artifacts` evita que el agente tenga que generar argumentos complejos y reduce errores.

Añadimos una serie de restricciones explícitas en el prompt del agente para asegurar un funcionamiento correcto y evitar alucinaciones, bucles o formatos compatibles con PDF, obligándolo a:

- No inventar bibliografía.
- Utilizar únicamente la salida de las herramientas.
- Generar contenido en texto plano limpio.
- Evitar formatos problemáticos.

5. Problemas encontrados y soluciones

Durante el desarrollo se identificaron varios problemas relevantes:

- El agente no generaba el PDF:
Se solucionó forzando la llamada a una única herramienta final (`save_report_artifacts`).
- Generación de rutas incorrectas:
El modelo inventaba rutas como `/tmp/....`. Se corrigió obligando a usar exclusivamente las rutas devueltas por la herramienta.
- Formato incorrecto del contenido:
Se detectó la aparición de tablas Markdown, símbolos LaTeX y secuencias como `"\n"`. Se solucionó mediante el método `clean_text`.

6. Resultados

El sistema desarrollado permite generar automáticamente:

- Un informe técnico estructurado en PDF,
- Un archivo JSON con estadísticas del documento,
- Una respuesta final con las rutas correctas de los archivos generados.

El comportamiento del agente es consistente y cumple con los requisitos de la práctica, tanto en estructura como en uso de herramientas.

7. Conclusiones

El agente implementado cumple con los objetivos planteados, proporcionando una solución completa para la generación automática de informes técnicos a partir de información web. La principal mejora del diseño ha sido la separación clara entre la lógica del agente y la generación de artefactos, lo que permite un flujo más controlado y fiable.